

**Merge Conflict
Programmer's Manual
CS450 – Spring 2025**

Table of Contents

C Files	1
comexec.c	1
comexec()	1
shutdown()	1
help()	2
version()	2
clear_screen()	3
return_color()	3
include/mpx/comexec.h	4
include/string.h	5
include/mpx/serial.h	6
include/mpx/device.h	10
include/sys_req.h	10
include/mpx/io.h	11
include/mpx/interrupts.h	12
include/conversions.h	13
include/ctype.h	14
include/memory.h	14
include/mpx/time.h	15
include/mpx/user_pcb.h	16
include/mpx/sys_call.h	17
include/mpx/alarm.h	18
include/mcb.h	19
 comhand.c	 22
comhand()	22
rOver_Startup()	22
include/mpx/comexec.h	22
include/mpx/device.h	24
include/mpx/serial.h	24

include/sys_req.h	27
include/string.h	28
ctype.c	32
isdigit()	32
include/ctype.h	32
serial.c	33
buffer_refresh()	33
serial_open()	33
serial_close()	34
serial_read()	34
serial_write()	35
serial_interrupt()	35
serial_input_interrupt()	36
serial_output_interrupt()	36
get_dcb()	36
get_irq()	37
iocb_setup()	37
iocb_clear()	38
include/serial.h	38
include/mpx/io.h	42
include/string.h	42
include/sys_req.h	44
include/conversions.h	45
include/mpx/comexec.h	46
conversions.c	48
itoa()	48
bcdToChar()	48
intToBCD()	49
Math_power()	49

Hex2Dec()	49
Dec2Hex()	50
include/conversions.h	50
include/string.h	51
include/ctype.h	53
string.c	54
contains()	54
substr()	54
isNumeric()	55
charCount()	55
indexOf()	55
print()	56
println()	56
print_error()	57
print_color()	57
trim()	58
clear()	58
strcat()	59
user_pcb.c	60
create_PCB()	60
delete_PCB()	60
block_PCB()	61
unblock_PCB()	61
suspend_PCB()	62
resume_PCB()	62
set_PCB_priority()	63
show_PCB()	63
show_ready_PCB()	64
show_blocked_PCB()	64
show_all_PCB()	65

Load_R3()	65
Load_R3_Sus()	66
include/mpx/pcb.h	66
include/mpx/user_pcb.h	68
include/sys_req.h	69
include/conversions.h	70
include/string.h	71
includes/mpx/comexec.h	74
time.c	76
get_time()	76
set_time()	76
get_date()	77
set_date()	77
get_hour()	78
get_minute()	78
get_second()	79
include/sys_req.h	79
include/conversions.h	80
include/mpx/io.h	81
include/string.h	81
include/mpx/interrupts.h	84
include/conversions.h	85
pcb.c	87
return_queue()	87
pcb_setup()	87
pcb_allocate()	88
pcb_free()	88
pcb_find()	89
pcb_insert()	89
pcb_remove()	90

includes/mpx/pcb.h	90
includes/memory.h	92
includes/string.h	92
includes/context.h	94
 sys_call.c	 96
sys_call()	96
find_first_ready()	96
IO_Scheduler()	97
IO_Completion()	97
includes/mpx/sys_call.h	98
includes/context.h	98
includes/mpx/pcb.h	99
 alarm.c	 102
alarm_process()	102
create_alarm()	102
includes/mpx/time.h	103
includes/mpx/pcb.h	104
includes/sys_req.h	105
includes/string.h	106
includes/mpx/sys_call.h	108
 mcb.c	 110
initialize_heap()	110
mcb_setup()	111
alloc_mcb()	111
return_list()	112
allocate_memory()	112
free_memory()	112
mcb_insert()	113
mcb_remove()	113

mcb_find()	114
includes/string.h	114
includes/memory.h	116
includes/mpx/comexec.h	116
includes/conversions.h	117
user_mcb.c	119
show_mcb()	119
show_alloc_mem()	119
show_free_mem()	120
includes/string.h	120
includes/memory.h	122
includes/mpx/comexec.h	122
includes/conversions.h	123
includes/mcb.h	124
Assembly Files	128
sys_call_isr.s	128
serial_isr.s	129

C Files

comexec.c

- Comexec.c is where the execution of all user entered commands occurs. It holds functionality for version, help, get_time, get_date, set_time, set_date, and shutdown.

comexec()

- int comexec(char buf[])
 - Authors: Tanner Forbes, Chris Jones, Izaak Whetsell, Evan Humphrey
 - This function executes the different system commands after reading the buffer in from comhand()
 - Parameters:
 - char buf[] is an array of characters that the user has typed
 - Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

shutdown()

- int shutdown(void)
 - Authors: Izaak Whetsell, Chris Jones
 - This function terminates the command handling sequence
 - Parameters:
 - None

- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

help()

- int help(void)
- Author: Chris Jones
- This function prints a list of each available command
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

version()

- int version(void)
- Authors: Chris Jones, Evan Humphrey
- This function prints the current version of the system
- Parameters:
 - None
- Return Values:

- This function returns an integer that represents the status of the functions completion.
- 1 is to shutdown the operating system
- 0 is to continue running

clear_screen()

- `int clear_screen(void)`
- Authors: Evan Humphrey
- This function clears the window being viewed in the terminal and returns the cursor to the "home" position
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 0 for success

return_color()

- `char* return_color(void)`
- Authors: Chris Jones
- This function returns the desired color code saved into a special variable
- Parameters:
 - None
- Return Values:
 - This function returns a string that holds the color code requested

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Global Variables:

```
char color[10]
```

This variable tracks the color set by the user for their CLI theme

Function Prototypes:

```
int comexec(char buf[])
```

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

```
int shutdown(void)
```

- This function will terminate the command handling sequence in the system and shut down any processes executing

```
int help(void)
```

- This function will print a list of each available command from the current point in the system

```
int version(void)
```

- This function prints the compilation date and version of the system

```
int clear_screen(void)
```

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

- This function returns the desired color requested by a special variable in the program

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

`int isNumeric(char* str)`

- This function determines whether a string is entirely numeric

`int charCount(char* str, char c)`

- This function finds the number of occurrences of a specific character in a given string

`int indexOf(char* str, char searched_char)`

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

char* strcat(char* des, const char* src)

- This function concatenates two strings together into a destination string

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRIAGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Structs:

- **dcb**
 - device dev_address
 - int dev
 - int open
 - int status
 - int* event_flag
 - char* input_buf
 - size_t input_len
 - size_t input_count
 - char* output_buf
 - size_t output_len
 - size_t output_count
 - char ring_buffer[MAX_RING_BUFFER_SIZE]
 - int ring_start
 - int ring_end
 - int ring_count
 - struct iocb* queue_head
- **iocb**
 - struct pcb* process
 - int operation

- char* buffer
- size_t length
- size_t transferred
- int even_flag
- struct iocb* next

Lists:

- dcb* dcb_list[4];

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

int serial_open(device dev, int speed)

- This function opens the desired serial port for communication

int serial_close(device dev)

- This function closes the desired serial port off from communication

int serial_read(device dev, char *buf, size_t len)

- This function performs character transfer from the device to the system's input buffer

int serial_write(device dev, char *buf, size_t len)

- This function performs transfer from the system's serial port and outputs the contents to the user

void serial_interrupt(void)

- This function raises an interrupt whenever a serial operation occurs in the system

void serial_input_interrupt(dcb* dev)

- This function raises an interrupt for input into the system

void serial_output_interrupt(dcb* dev)

- This function raises an interrupt whenever the system needs to print output to the user

dcb* get_dcb(int devno)

- This function returns a pointer to the desired DCB

int get_irq(int devno)

- This function returns the IRQ number for a given device

**iocb* iocb_setup(struct pcb* current_pcb, char* new_buffer,
size_t new_length, int op_code)**

- This function returns a pointer to a newly created IOCB initialized with the given parameters

void iocb_clear(struct iocb* cur_iocb)

- This function clears the contents of the given IOCB and frees its memory

include/mpx/device.h

This header file contains the constants used to identify the device I/O ports in the system

Enums:

device

COM1,

COM2,

COM3,

COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/sys_req.h

This file contains all system request functions and constants

Enums:

op_code

EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- INVALID_OPERATION
- INVALID_BUFFER
- INVALID_COUNT

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`include/mpx/io.h`

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

`outb(port, data)`

- This will take one byte of data and write it to an I/O port

`inb(port, data)`

- This will take one byte of data and read it into an I/O port

include/mpx/interrupts.h

This file contains kernel functions related to software and hardware interrupts in the system

Constants include:

- cli()
- sti()

These constants are used to disable and enable interrupts in the system respectively

Function Prototypes:

void irq_init(void)

- This function installs the first 32 interrupt handlers into the IRQ (Interrupt Request Table)

void pic_init(void)

- This function initializes the programmable interrupt controllers and performs the necessary remapping of IRQs, it leaves interrupts turned off.

void idt_init(void)

- This function creates and installs the interrupt descriptor table

void idt_install(int vector, void (*handler)(void *))

- The function installs an interrupt handler

include/conversions.h

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

- This function takes an integer value and returns a char literal representation of its binary equivalent

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:

int isspace(int c)

- This function determines if the given character is a whitespace

int isdigit(char c)

- This function determines if a given character is a digit

include/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:

void *sys_alloc_mem(size_t size)

- This function dynamically allocates memory in the system

int sys_free_mem(void *ptr)

- This function frees dynamically allocated memory located within the system

void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))

- This function installs user-supplied heap-management functions

include/mpx/time.h

This file contains all functions relating to the real time clock (RTC)

Function Prototypes:

int get_time(void)

- This function prints out the current time in Eastern Standard Time using the real time clock

int set_time(char buf[])

- This function utilizes the real time clock registers to set the time in the system

int get_date(void)

- This function retrieves the date using the real time clock and prints it to the terminal

int set_date(char buf[])

- This function utilizes the real time clock registers to set the current date within the system

int get_hour(void)

- This function returns an integer representation of the current hour in the system.

int get_minute(void)

- This function returns an integer representation of the current minute in the system.

int get_second(void)

- This function returns an integer representation of the current second in the system.

include/mpx/user_pcb.h

This file contains all user functions related to the creation, alteration, deletion, and display of all process control blocks.

Function Prototypes:

int create_PCB(char* name, int class, int priority)

- This function creates a new process and assigns values into it based on its given parameters

int delete_PCB(char* name)

- This function deletes a PCB from memory

int block_PCB(char* name)

- This function edits a given process to be put into a blocked state

int unblock_PCB(char* name)

- This function edits a given process to be placed into a ready state

int suspend_PCB(char* name)

- This function edits a given process to be placed into a suspended state

int resume_PCB(char* name)

- This function edits a given process to be placed into a not suspended state

int set_PCB_priority(char* name, int priority)

- This function takes a priority and edits a given process to change to the specified priority

int show_PCB(char* name)

- This function displays a given process block and its information given its name

int show_ready_PCB(void)

- This function displays all ready processes and their associated information into the terminal window

int show_blocked_PCB(void)

- This function displays all blocked processes and their associated information into the terminal window

int show_all_PCB(void)

- This function displays all existing processes from the system into the terminal window

int Load_R3(void)

- This function deploys 5 new processes to test the dispatching method of the R3 module implementation.

int Load_R3_Sus(int pcb_num, int priority)

- This function loads a ready, suspended process into the system with a numbered name specified in the function.

includes/mpx/sys_call.h

This file contains all functions pertaining to process dispatching.

Global Variables:

pcb* CURRENT_PCB

This pointer points to the currently running process in the system, and is updated during every context switch.

`ctx* GLOBAL_CTX`

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

`pcb* nextPCB`

This pointer is used to track the position of the next process in the queue.

`struct context* sys_call(context* ctx)`

- This function returns a pointer to the next process to execute in the system.

`pcb* find_first_ready(void)`

- This functions finds and returns the first process in the ready queue.

`includes/mpx/alarm.h`

This file contains all functions pertaining to alarm creation and management.

`void alarm_process(void)`

- This function monitors the alarm processes that are contained within the ready queue.

`int create_alarm(char* message, char* time)`

- This function takes the parameters needed from the user and creates an alarm process to be inserted into the queue.

include/mcb.h

This file contains all functions that execute kernel level commands relating to Memory Control Blocks

Structs:

- **mcb**
 - unsigned int start_addr
 - int size
 - struct mcb* next_node
 - struct mcb* prev_node
 - int status
 - char* name

- **list**
 - mcb* head
 - pcb* tail

Global Variables:

- list MCB_LIST contains all working MCBs in the system
- char str[100] is the string buffer for printing with itoa

Function Prototypes:

void initialize_heap(size_t size)

- This function is called to allocate the desired amount of bytes contiguously in the system for the heap

mcb* mcb_setup(unsigned int start_addr, int size)

- This function takes in multiple parameters for a desired memory control block, allocates memory for it, and returns a pointer to the newly initialized block with all desired values

`void * allocate_memory(size_t size)`

- This function will allocate a desired chunk of memory in the heap and assign an MCB to mark it. This function also combines any free blocks in the system to preserve more memory for incoming processes

`list* return_list(void)`

- This function returns a pointer to the head of the MCB list

`mcb* alloc_mcb(void)`

- This function allocates memory for a new MCB

`int free_memory(void *addr)`

- This function frees a block of allocated memory at the desired address

`void mcb_insert( mcb* new_mcb)`

- This function inserts a new MCB into the list

`void mcb_remove( mcb* current_mcb)`

- This function removes the desired MCB from the list

`mcb* mcb_find(unsigned int start_addr)`

- This function locates the desired MCB in the list using its start address

`void show_mcb(mcb* mcb)`

- This function prints out all details pertaining to the desired MCB you give a reference to

```
void show_alloc_mem(void)
```

- This function prints out all allocated memory blocks in the heap

```
void show_free_mem(void)
```

This function prints out all free memory blocks in the heap

comhand.c

- This file contains the command handler, which scans for any commands entered into the system

`comhand()`

- `void comhand(void)`
- Authors: Izaak Whetsell
- This function scans for commands to execute in the system which passes them on to be executed in the system
- Parameters:
 - None
- Return Values:
 - None

`r0ver_Startup()`

- `void r0ver_Startup(void)`
- Authors: Izaak Whetsell
- This function prints the OS startup image when the system is loaded
- Parameters:
 - None
- Return Values:
 - None

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Function Prototypes:

`int comexec(char buf[])`

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

`int shutdown(void)`

- This function will terminate the command handling sequence in the system and shut down any processes executing

`int help(void)`

- This function will print a list of each available command from the current point in the system

`int version(void)`

- This function prints the compilation date and version of the system

`int clear_screen(void)`

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

- This function returns the desired color requested by a special variable in the program

include/mpx/device.h

This header file contains the constants used to identify the device I/O ports in the system

Enums:

device

COM1,

COM2,

COM3,

COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRIAGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Structs:

- dcb

- device dev_address
- int dev
- int open
- int status
- int* event_flag
- char* input_buf
- size_t input_len
- size_t input_count
- char* output_buf
- size_t output_len
- size_t output_count
- char ring_buffer[MAX_RING_BUFFER_SIZE]
- int ring_start
- int ring_end
- int ring_count
- struct iocb* queue_head

- **iocb**
 - struct pcb* process
 - int operation
 - char* buffer
 - size_t length
 - size_t transferred
 - int even_flag
 - struct iocb* next

Lists:

- dcb* dcb_list[4];

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

int serial_open(device dev, int speed)

- This function opens the desired serial port for communication

int serial_close(device dev)

- This function closes the desired serial port off from communication

int serial_read(device dev, char *buf, size_t len)

- This function performs character transfer from the device to the system's input buffer

int serial_write(device dev, char *buf, size_t len)

- This function performs transfer from the system's serial port and outputs the contents to the user

void serial_interrupt(void)

- This function raises an interrupt whenever a serial operation occurs in the system

void serial_input_interrupt(dcb* dev)

- This function raises an interrupt for input into the system

void serial_output_interrupt(dcb* dev)

- This function raises an interrupt whenever the system needs to print output to the user

dcb* get_dcb(int devno)

- This function returns a pointer to the desired DCB

int get_irq(int devno)

- This function returns the IRQ number for a given device

iocb* iocb_setup(struct pcb* current_pcb, char* new_buffer, size_t new_length, int op_code)

- This function returns a pointer to a newly created IOCB initialized with the given parameters

void iocb_clear(struct iocb* cur_iocb)

- This function clears the contents of the given IOCB and frees its memory

include/sys_req.h

This file contains all system request functions and constants

Enums:

op_code

EXIT,

IDLE,

READ,

WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

int sys_req(op_code op, ...)

- This function requests a kernel operation within the system to perform an action based on the input parameters.

include/string.h

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:

void* memcpy(void * restrict dst, const void * restrict src, size_t n)

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

void* memset(void *address, int c, size_t n)

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

`char* strcat(char* des, const char* src)`

- This function concatenates two strings together into a destination string

ctype.c

- ctype.c holds functions relating to the manipulation or extraction of data of the char type.

isdigit()

- int isdigit(char c)
- Author: Chris Jones
- This function determines whether a given character is a digit
- Parameters:
 - char c is the character that is being judged to be or not to be an integer
- Return Values:
 - This function returns an int that represents a boolean.
 - 1 is true and the character is a digit
 - 0 is false and the character is not a digit

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:

int isdigit(char c)

- This function determines if a given character is a numeric digit and returns an integer code of 1 if the character is a digit, and a 0 if not

serial.c

- serial.c holds methods and structures pertaining to the input and output of serial communications.

buffer_refresh()

void buffer_refresh(char *buffer, int buf_length, int pos)

- Author: Evan Humphrey
- This function keeps track of the buffer in the terminal window as keys are pressed, refreshing every time the buffer is updated.
- Parameters:
 - char *buffer is a string of current characters typed by the user
 - int buf_length is the length of the passed buffer
 - int pos is the position of the user's cursor
- Return Values:
 - None

serial_open()

int serial_open(device dev, int speed)

- Author: Chris Jones
- This function opens the desired device serial port for communication
- Parameters:
 - Device dev is the device to open the serial port on
 - int speed is the baud rate for the serial port
- Return Values:

- Returns an integer code
 - 0 on success
 - Non-zero on failure

serial_close()

int serial_close(device dev)

- Author: Chris Jones
- This function keeps closes the serial communication port on the specified device
- Parameters:
 - device dev is the device to close the serial port on
- Return Values:
 - Returns an integer code
 - 0 on success
 - Non-zero on failure

serial_read()

int serial_read(device dev, char *buf, size_t len)

- Author: Evan Humphrey
- This function performs a character transfer from the specified devices serial port into the systems buffer
- Parameters:
 - device dev is the device to read input from
 - char* buf buffer to copy the input into
 - size_t len is the length of the provided buffer
- Return Values:
 - Returns a non-zero integer on failure

- Returns the length of copied characters in the system on success

serial_write()

int serial_write(device dev, char *buf, size_t len)

- Author: Evan Humphrey
- This function performs a character transfer from the systems buffer and prints the contents to the user
- Parameters:
 - device dev is the device to read input from
 - char* buf buffer to copy the input into
 - size_t len is the length of the provided buffer
- Return Values:
 - Returns a non-zero integer on failure
 - Returns the length of copied characters outputted from the system on success

serial_interrupt()

void serial_interrupt(void)

- Author: Izaak Whetsell
- This function raises an interrupt in the system when an I/O event is triggered
- Parameters:
 - None
- Return Values:
 - None

serial_input_interrupt()
void serial_input_interrupt(dcb* dev)

- Author: Izaak Whetsell
- This function raises an interrupt whenever the device needs to read characters from its serial port
- Parameters:
 - dcb* dev is the device to read characters from
- Return Values:
 - None

serial_output_interrupt()
void serial_output_interrupt(dcb* dev)

- Author: Izaak Whetsell
- This function raises an interrupt whenever the device needs to copy characters from its buffer to output to the user
- Parameters:
 - dcb* dev
- Return Values:
 - None

get_dcb()
dcb* get_dcb(int devno)

- Author: Chris Jones
- This function returns a pointer to the specified DCB given its device number

- Parameters:
 - int devno is the number corresponding to each device
- Return Values:
 - A pointer to the requested device control block

get_irq()

int get_irq(int devno)

- Author: Chris Jones
- This function returns the IRQ number for the given device
- Parameters:
 - int devno is the device to find
- Return Values:
 - The IRQ value for the specified device

iocb_setup()

iocb* iocb_setup(struct pcb* current_pcb, char* new_buffer, size_t new_length, int op_code)

- Author: Tanner Forbes
- This function returns a shiny new IOCB initialized with the given parameters
- Parameters:
 - struct pcb* is the process to associate the I/O operation with
 - char* new_buffer is the buffer associated with the I/O operation
 - size_t new_length is the size of the given buffer

- int op_code is the operation to be performed by the IOCB
- Return Values:
 - A pointer to the newly created IOCB struct

iocb_clear()

void iocb_clear(struct iocb* cur_iocb)

- Author: Tanner Forbes
- This function clears the contents of the struct and frees the memory of the struct
- Parameters:
 - Struct iocb* cur_iocb is the pointer to the IOCB to remove
- Return Values:
 - None

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE

- CARRAIGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Structs:

- **dcb**
 - device dev_address
 - int dev
 - int open
 - int status
 - int* event_flag
 - char* input_buf
 - size_t input_len
 - size_t input_count
 - char* output_buf
 - size_t output_len
 - size_t output_count
 - char ring_buffer[MAX_RING_BUFFER_SIZE]
 - int ring_start
 - int ring_end
 - int ring_count
 - struct iocb* queue_head
- **iocb**
 - struct pcb* process
 - int operation
 - char* buffer
 - size_t length
 - size_t transferred
 - int even_flag
 - struct iocb* next

Lists:

- dcb* dcb_list[4];

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

int serial_open(device dev, int speed)

- This function opens the desired serial port for communication

int serial_close(device dev)

- This function closes the desired serial port off from communication

int serial_read(device dev, char *buf, size_t len)

- This function performs character transfer from the device to the system's input buffer

int serial_write(device dev, char *buf, size_t len)

- This function performs transfer from the system's serial port and outputs the contents to the user

void serial_interrupt(void)

- This function raises an interrupt whenever a serial operation occurs in the system

void serial_input_interrupt(dcb* dev)

- This function raises an interrupt for input into the system

void serial_output_interrupt(dcb* dev)

- This function raises an interrupt whenever the system needs to print output to the user

dcb* get_dcb(int devno)

- This function returns a pointer to the desired DCB

int get_irq(int devno)

- This function returns the IRQ number for a given device

iocb* iocb_setup(struct pcb* current_pcb, char* new_buffer, size_t new_length, int op_code)

- This function returns a pointer to a newly created IOCB initialized with the given parameters

`void iocb_clear(struct iocb* cur_iocb)`

- This function clears the contents of the given IOCB and frees its memory

`include/mpx/io.h`

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

`outb(port, data)`

- This will take one byte of data and write it to an I/O port

`inb(port, data)`

- This will take one byte of data and read it into an I/O port

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:

`void* memcpy(void * restrict dst, const void * restrict src, size_t n)`

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

void* memset(void *address, int c, size_t n)

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

include/sys_req.h

This file contains all system request functions and constants

Enums:

op_code

EXIT,

IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- INVALID_OPERATION
- INVALID_BUFFER
- INVALID_COUNT

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`include/conversions.h`

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

- This function takes an integer value and returns a char literal representation of its binary equivalent

`include/mpx/comexec.h`

This file contains all functions that execute system commands when called in the terminal

Global Variables:

`char color[10]`

This variable tracks the color set by the user for their CLI theme

Function Prototypes:

`int comexec(char buf[])`

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

`int shutdown(void)`

- This function will terminate the command handling sequence in the system and shut down any processes executing

`int help(void)`

- This function will print a list of each available command from the current point in the system

`int version(void)`

- This function prints the compilation date and version of the system

`int clear_screen(void)`

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

- This function returns the desired color requested by a special variable in the program

conversions.c

- conversions.c contains all functions that are contained in C standard library functions

itoa()

- char* itoa(int num, char* str, int base)
- Author: Tanner Forbes
- This function converts an integer literal into an ASCII character based on the ASCII table of characters and values
- Parameters:
 - int num is a number to be converted to a character
 - char* str is a where the new character digit will be stored
 - int base is the base of the number system used
- Return Values:
 - char* is the newly converted digit as a character

bcdToChar()

- char bcdToChar(unsigned char bcd)
- Author: Tanner Forbes
- This function converts a Binary Coded Decimal (BCD) value to a character
- Parameters:
 - unsigned char bcd is the Binary Coded Decimal value to be turned into a character
- Return Values:

- Char is the newly converted BCD as a character

intToBCD()

- unsigned int intToBCD(unsigned int num)
- Author: Chris Jones
- This function converts an integer to a Binary Coded Decimal (BCD) value
- Parameters:
 - unsigned int num is the number to convert to BCD.
- Return Values:
 - Unsigned int is the BCD value of the passed unsigned integer

Math_power()

- int Math_power(int num, int exponent)
- Author: Tanner Forbes
- This function calculates the specified power of an integer
- Parameters:
 - int num is the number to put to a power
 - int exponent is the power to raise the number to
- Return Values:
 - This function returns the answer to the specified function as an integer

Hex2Dec()

- unsigned int Hex2Dec(char *hex, int hex_size)

- Author: Tanner Forbes
- This function converts a hexadecimal value into an integer
- Parameters:
 - char* hex is the hexadecimal number to convert
 - int hex_size is the size (in chars) of the hexadecimal string
- Return Values:
 - This function returns the integer representation of the hex number

Dec2Hex()

- char* Dec2Hex(unsigned int dec)
- Author: Tanner Forbes
- This function converts an integer to a hexadecimal string value
- Parameters:
 - unsigned int dec is the number to convert to hex.
- Return Values:
 - This function returns a string literal representation of the hexadecimal representation of the passed in integer

include/conversions.h

This file contains a subset of C standard functions

Function Prototypes:

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

include/string.h

This file contains all functions pertaining to string manipulation and identification

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

char* strcat(char* des, const char* src)

- This function concatenates two strings together into a destination string

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:

`int isspace(int c)`

- This function determines if the given character is a whitespace

`int isdigit(char c)`

- This function determines if a given character is a digit

string.c

- string.c holds functions relating to the manipulation or extraction of data of strings.

contains()

- int contains(const char *str1, const char *str2)
- Authors: Tanner Forbes and Chris Jones
- Checks to see if a string is inside of another string
- Parameters:
 - const char *s1 is the phrase to check inside of
 - const char *s2 is the string to find
- Return Values:
 - int will be 1 if the second string was located inside the first and 0 if the string was not found or contained in the first.

substr()

- char* substr(char *str,int index)
- Author: Chris Jones
- This function obtains the substring of a string starting at the passed index and proceeding to the end of the string
- Parameters:
 - char *str is the passed string
 - int index is the index to begin the substring
- Return Values:
 - Char * is the substring starting at the passed index and ending with the last character of the string

isNumeric()

- `int isNumeric(char *str)`
- Author: Chris Jones
- Checks to see if the passed string is an assortment of only digit character
- Parameters:
 - `char *str` is the string to check for digits
- Return Values:
 - `int` will be 1 if the passed string is a digit and 0 if any character is not a digit

charCount()

- `int charCount(char str, char c)`
- Author: Chris Jones
- Counts the number of occurrences of the character `c` in a string
- Parameters:
 - `char *str` is the string of characters to count
 - `char c` is the character to count at each occurrence in the string
- Return Values:
 - `int` is the number of occurrences of the character in the string

indexOf()

- `int indexOf(char *str, char searched_char)`

- Author: Tanner Forbes
- Attempts to find the index of a specified character in a string.
- Parameters:
 - char *str is the string to check for the characters position
 - char searched_char is the character to find the position of
- Return Values:
 - int is the index of the found character or -1 for the character not being in the string

print()

- void print(const char* sentence)
- Author: Izaak Whetsell
- Prints a desired string to the terminal
- Parameters:
 - const char* sentence is the string to be printed in the terminal
- Return Values:
 - There are no return values

println()

- void println(void)
- Author: Izaak Whetsell

- Prints a newline to the terminal
- Parameters:
 - There are no parameters for this function
- Return Values:
 - There are no return values

`print_error()`

- `void print_error(const char* sentence)`
- Author: Izaak Whetsell
- Prints the text in the terminal as red to indicate an error has occurred.
- Parameters:
 - `const char* sentence` is the string to be printed in the terminal
- Return Values:
 - There are no return values

`print_color()`

- `void print_color(const char* sentence, const char* color_code)`
- Author: Evan Humphrey
- Prints a desired string to the terminal in the color specified. Available options are green, magenta, blue, yellow, and cyan
- Parameters:

- `const char* sentence` is the string to be printed in the terminal
- `const char* color_code` is the identifier of what color to print to the terminal.
- Return Values:
 - There are no return values

`trim()`

- `void trim (char sentence[])`
- Author: Izaak Whetsell
- This function trims the whitespace off the given string on each end
- Parameters:
 - `char sentence[]` is the string to be trimmed
- Return Values:
 - Returns the pointer to the trimmed string

`clear()`

- `void clear(char* arr)`
- Authors: Chris Jones
- Clears a string by setting all characters to 0
- Parameters:
 - `Char* arr` is the string to be cleared
- Return Values:
 - None

strcat()

- `char* strcat(char* des, const char* src)`
- Authors: Tanner Forbes
- Concatenates two given strings
- Parameters:
 - `char* des` is the string to be concatenated into
 - `const char* src` is the string to concatenate from
- Return Values:
 - This function returns the concatenated string

user_pcb.c

- user_pcb.c contains all functions that user functions rely on to create, edit, and display process information.

`create_PCB()`

- `int create_PCB(char* name, int class, int priority)`
- Author: Chris Jones
- Creates a process control block by allocating memory and setting the fields to the provided parameters.
- Parameters
 - `char* name` is the name set in the created PCB
 - `int class` is the class that the PCB will be set to (0 for a user process, 1 for a system process)
 - `int priority` is the priority number to be assigned to the PCB (process priority is within the range of 0-9, highest to lowest, respectively)
- Return Values:
 - `int` is the status code of the operation, 0 on success and -1 for an invalid state

`delete_PCB()`

- `int delete_PCB(char* name)`
- Author: Chris Jones
- Deletes a specified process control block from memory.
- Parameters
 - `char* name` is the name to search for in the process queue to delete

- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

block_PCB()

- int block_PCB(char* name)
- Author: Chris Jones
- Sets the state of the desired process into a blocked state from the current state it is in, if the process is already blocked, the user will be notified and nothing will be changed.
- Parameters
 - char* name is the name to search for in the process queue to edit
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

unblock_PCB()

- int unblock_PCB(char* name)
- Author: Chris Jones
- Sets the state of the desired process into an unblocked state from the current state it is in, if the process is already unblocked, the user will be notified and nothing will be changed.
- Parameters
 - char* name is the name to search for in the process queue to edit

- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

suspend_PCB()

- int suspend_PCB(char* name)
- Author: Chris Jones
- Sets the state of the desired process into a suspended state from the current state it is in, if the process is already suspended, the user will be notified and nothing will be changed.
- Parameters
 - char* name is the name to search for in the process queue to edit
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

resume_PCB()

- int resume_PCB(char* name)
- Author: Chris Jones
- Sets the state of the desired process into a ready state from the current state it is in, if the process is already ready, the user will be notified and nothing will be changed.
- Parameters

- char* name is the name to search for in the process queue to edit
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

set_PCB_priority()

- int set_PCB_priority(char* name, int priority)
- Author: Chris Jones
- Sets the priority of the desired process to the given priority.
- Parameters
 - char* name is the name set in the created PCB
 - int priority is the priority number to be assigned to the PCB (process priority is within the range of 0-9, highest to lowest, respectively)
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

show_PCB()

- int show_PCB(char* name)
- Author: Chris Jones
- Displays the desired PCB and all of the information contained within it.
- Parameters

- char* name is the name to search for in the process queue to display
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

show_ready_PCB()

- int show_ready_PCB(void)
- Author: Chris Jones
- Displays the ready queue and all processes within it, as well as all of the information contained in each respective process
- Parameters
 - This function has no parameters
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

show_blocked_PCB()

- int show_blocked_PCB(void)
- Author: Chris Jones
- Displays the blocked queue and all processes within it, as well as all of the information contained in each respective process
- Parameters
 - This function has no parameters

- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

`show_all_PCB()`

- int show_all_PCB(void)
- Author: Chris Jones
- Displays the blocked and ready queues and all processes within them, as well as all of the information contained in each respective process
- Parameters
 - This function has no parameters
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

`Load_R3()`

- int Load_R3(void)
- Author: Izaak Whetsell
- Creates 5 new test rprocesses to test the dispatching implementation of the R3 module of MPX.
- Parameters
 - This function has no parameters
- Return Values:
 - int is the status code of the operation, 0 on success and -1 for an invalid state

Load_R3_Sus()

- `int Load_R3_Sus(int pcb_num, int priority)`
- Author: Izaak Whetsell
- Creates a ready, suspended process and places it into the suspended queue in the system.
- Parameters
 - `int pcb_num` is the numbered name of the process to be created
- Return Values:
 - `int` is the status code of the operation, 0 on success and -1 for an invalid state

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:

pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,
BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
 - `char* name`

- int class
- int priority
- int state
- unsigned char* stack
- unsigned char* stack_location
- struct pcb* next_node
- struct pcb* prev_node
- **queue**
 - int queue_type
 - pcb* head
 - pcb* tail

Function Prototypes:

queue* return_queue(int class)

- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)

- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)

- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)

- This function frees up the memory allocated to a process control block

pcb* pcb_find(const char * name)

- This function finds a given process in the system in either queue

void pcb_insert(pcb* pcbPtr)

- This function inserts a new PCB into its corresponding queue in the system

`int pcb_remove(pcb* pcbPtr)`

- This function removes the specified PCB from the queue but does not delete it from the system

`includes/mpx/user_pcb.h`

This file contains all user functions related to the creation, alteration, deletion, and display of all process control blocks.

Function Prototypes:

`int create_PCB(char* name, int class, int priority)`

- This function creates a new process and assigns values into it based on its given parameters

`int delete_PCB(char* name)`

- This function deletes a PCB from memory

`int block_PCB(char* name)`

- This function edits a given process to be put into a blocked state

`int unblock_PCB(char* name)`

- This function edits a given process to be placed into a ready state

`int suspend_PCB(char* name)`

- This function edits a given process to be placed into a suspended state

`int resume_PCB(char* name)`

- This function edits a given process to be placed into a not suspended state

`int set_PCB_priority(char* name, int priority)`

- This function takes a priority and edits a given process to change to the specified priority

`int show_PCB(char* name)`

- This function displays a given process block and its information given its name

`int show_ready_PCB(void)`

- This function displays all ready processes and their associated information into the terminal window

`int show_blocked_PCB(void)`

- This function displays all blocked processes and their associated information into the terminal window

`int show_all_PCB(void)`

- This function displays all existing processes from the system into the terminal window

`int Load_R3(void)`

- This function deploys 5 new processes to test the dispatching method of the R3 module implementation.

`int Load_R3_Sus(int pcb_num, int priority)`

- This function loads a ready, suspended process into the system with a numbered name specified in the function.

`includes/sys_req.h`

This file contains all system request functions and constants

Enums:

`op_code`

EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- INVALID_OPERATION
- INVALID_BUFFER
- INVALID_COUNT

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`includes/conversions.h`

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

- This function takes an integer value and returns a char literal representation of its binary equivalent

`includes/string.h`

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:

`void* memcpy(void * restrict dst, const void * restrict src, size_t n)`

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

`void* memset(void *address, int c, size_t n)`

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

char* strcat(char* des, const char* src)

- This function concatenates two strings together into a destination string

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Global Variables:

```
char color[10]
```

This variable tracks the color set by the user for their CLI theme

Function Prototypes:

```
int comexec(char buf[])
```

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

```
int shutdown(void)
```

- This function will terminate the command handling sequence in the system and shut down any processes executing

```
int help(void)
```

- This function will print a list of each available command from the current point in the system

```
int version(void)
```

- This function prints the compilation date and version of the system

```
int clear_screen(void)
```

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

- This function returns the desired color requested by a special variable in the program

time.c

- time.c contains all functions that can retrieve or manipulate time/date functions within the system using the real-time clock (RTC)

get_time()

- int get_time(void)
- Author: Tanner Forbes
- This function utilizes the real time clock registers to display the current time
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

set_time()

- int set_time(char buf[])
- Author: Chris Jones
- This function utilizes the real time clock registers to set the current time in the system
- Parameters:
 - char buf[] is an array of characters that the user has typed. This array should contain the proper time format (HH:MM:SS)

- Valid Parameters:
 - Valid hour values include: 01-24
 - Valid minute values include: 00-59
 - Valid second values include: 00-59
- All valid entries for single digits MUST include a beginning 0 before the parameter
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

`get_date()`

- `int get_date(void)`
- Author: Tanner Forbes
- This function utilizes the real time clock registers to retrieve and display the current date
- Parameters:
 - None
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

`set_date()`

- `int set_date(char buf[])`
- Author: Tanner Forbes

- This function utilizes the real time clock registers to set the current date in the system
- Parameters:
 - char buf[] is an array of characters that the user has typed. This array should contain the proper date format (MM/DD/YYYY) and should accurately match a calendar.
 - Example requirements include no months greater than 12, no days greater than 31, each month has designated 28-30-31 day scheme, leap year, and year is within the 21st Century.
- Return Values:
 - This function returns an integer that represents the status of the functions completion.
 - 1 is to shutdown the operating system
 - 0 is to continue running

get_hour()

- int get_hour(void)
- Author: Tanner Forbes
- This function utilizes the real time clock registers to return the current hour
- Parameters:
 - None
- Return Values:
 - This function returns an integer representation of the current hour in the system.

get_minute()

- `int get_minute(void)`
- Author: Tanner Forbes
- This function utilizes the real time clock registers to return the current minute
- Parameters:
 - None
- Return Values:
 - This function returns an integer representation of the current minute in the system.

`get_second()`

- `int get_second(void)`
- Author: Tanner Forbes
- This function utilizes the real time clock registers to return the current second
- Parameters:
 - None
- Return Values:
 - This function returns an integer representation of the current second in the system.

`includes/sys_req.h`

This file contains all system request functions and constants

Enums:

`op_code`

EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- INVALID_OPERATION
- INVALID_BUFFER
- INVALID_COUNT

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`includes/conversions.h`

This file contains a subset of C standard functions

Function Prototypes:

`int atoi(const char *s)`

- This function converts an ASCII string to an integer literal

`char* itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

This function converts a character into a Binary Coded Decimal Value (BCD)

`includes/mpx/io.h`

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

`outb(port, data)`

- This will take one byte of data and write it to an I/O port

`inb(port, data)`

- This will take one byte of data and read it into an I/O port

`includes/string.h`

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:

`void* memcpy(void * restrict dst, const void * restrict src, size_t n)`

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

`void* memset(void *address, int c, size_t n)`

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

`int strcmp(const char *s1, const char *s2)`

- This function compares two strings and returns an integer code specifying the result of the operation

`size_t strlen(const char *s)`

- This function returns the length of a passed-in string

`char* strtok(char * restrict s1, const char * restrict s2)`

- This function will tokenize a string and return

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

`int isNumeric(char* str)`

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

`char* strcat(char* des, const char* src)`

- This function concatenates two strings together into a destination string

`includes/mpx/interrupts.h`

This file contains kernel functions related to software and hardware interrupts in the system

Constants include:

- `cli()`
- `sti()`

These constants are used to disable and enable interrupts in the system respectively

Function Prototypes:

`void irq_init(void)`

- This function installs the first 32 interrupt handlers into the IRQ (Interrupt Request Table)

`void pic_init(void)`

- This function initializes the programmable interrupt controllers and performs the necessary remapping of IRQs, it leaves interrupts turned off.

`void idt_init(void)`

- This function creates and installs the interrupt descriptor table

`void idt_install(int vector, void (*handler)(void *))`

- The function installs an interrupt handler

include/conversions.h

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

- This function takes an integer value and returns a char literal representation of its binary equivalent

pcb.c

- pcb.c contains all kernel level functions pertaining to the manipulation of process control blocks and queues
- Queues:
 - BLOCKED
 - int queue_type: 1 (blocked)
 - pcb* head: NULL
 - pcb* tail: NULL
 - READY
 - int queue_type: 0 (ready)
 - pcb* head: NULL
 - pcb* tail: NULL

return_queue()

- queue* return_queue(int class)
- Author: Chris Jones
- This function returns a pointer to the head of the specified queue
- Parameters:
 - int class specifies what queue we would like to return
- Return Values:
 - This function returns a pointer to the desired queue

pcb_setup()

- pcb* pcb_setup(char* name, int class, int priority, int state, void (*function_ptr)(void))
- Author: Chris Jones & Izaak Whetsell

- This function allocates memory for a new process and sets each of its fields according to the specified parameters
- Parameters:
 - char* name is the name of the process
 - int class identifies the process as ready or blocked and suspended or not suspended
 - int priority sets the priority number of the process from highest to lowest (0-9)
 - void* function_ptr is the pointer to the process running that will be stored into it's context
- Return Values:
 - This function returns a pointer to the newly created process

pcb_allocate()

- pcb* pcb_allocate(void)
- Author: Tanner Forbes
- This function dynamically allocates memory for a new process and returns a pointer to it
- Parameters:
 - There are no parameters for this function
- Return Values:
 - This function returns a pointer to the dynamically allocated PCB

pcb_free()

- int pcb_free(struct pcb* free_pcb)
- Author: Tanner Forbes

- This function frees the memory used by a PCB
- Parameters:
 - Struct `pcb* free_pcb` is the pointer to the point in memory where we would like to free
- Return Values:
 - This function returns a status code based on the operation performed
 - 0 for success
 - 1 for an error

`pcb_find()`

- `pcb* pcb_find(const char * name)`
- Author: Tanner Forbes
- This function searches for a specific PCB in queue based on its name parameter
- Parameters:
 - `const char* name` is the name of the PCB we would like to find
- Return Values:
 - This function returns a pointer to the located PCB

`pcb_insert()`

- `void pcb_insert(pcb* pcbPtr)`
- Author: Izaak Whetsell
- This function inserts a PCB into its defined queue

- Parameters:
 - pcb* pcbPtr is the pointer to the PCB we would like to insert into a queue
- Return Values:
 - This function has no return values

pcb_remove()

- int pcb_remove(pcb* pcbPtr)
- Author: Tanner Forbes
- This function removes a PCB from the queue but does not delete it from the system.
- Parameters:
 - pcb* pcbPtr is the pointer to the PCB we would like to remove from the queue
- Return Values:
 - This function returns a status code based on the operation
 - 0 for success
 - 1 for an error

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:

pcb_state
 READY_SUS,
 READY_NOT_SUS,
 RUNNNING,

BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
 - char* name
 - int class
 - int priority
 - int state
 - unsigned char* stack
 - unsigned char* stack_location
 - struct pcb* next_node
 - struct pcb* prev_node

- **queue**
 - int queue_type
 - pcb* head
 - pcb* tail

Function Prototypes:

queue* return_queue(int class)

- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)

- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)

- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)

- This function frees up the memory allocated to a process control block

`pcb* pcb_find(const char * name)`

- This function finds a given process in the system in either queue

`void pcb_insert(pcb* pcbPtr)`

- This function inserts a new PCB into its corresponding queue in the system

`int pcb_remove(pcb* pcbPtr)`

This function removes the specified PCB from the queue but does not delete it from the system

`includes/memory.h`

This file contains system-specific dynamic memory functions

Function Prototypes:

`void *sys_alloc_mem(size_t size)`

- This function dynamically allocates memory in the system

`int sys_free_mem(void *ptr)`

- This function frees dynamically allocated memory located within the system

`void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))`

This function installs user-supplied heap-management functions

`includes/string.h`

This file contains all functions pertaining to string manipulation and identification

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

`void print_error(const char* sentence)`

- This function prints an error into the terminal window, this means that the text is printed out in a red color

`void print_color(const char* sentence, const char* color_code)`

- This function prints out text into the terminal into a variety of colors

`void trim(char sentence[])`

- This function trims the whitespace off the ends of the given string

`void clear(char* arr)`

- This function clears a string by setting all members to zero

`char* strcat(char* des, const char* src)`

- This function concatenates two strings together into a destination string

`includes/context.h`

This file contains all structures and functions relating to the context of a process.

Structs:

- **context**
 - Segment Registers:
 - int CS
 - int DS
 - int ES
 - int FS
 - int GS
 - int SS
 - Status Control Registers:

- int EIP
- int EFLAGS
- General Purpose Registers:
 - int EAX
 - int EBX
 - int ECX
 - int EDX
 - int ESI
 - int EDI
 - int EBP
 - int ESP

sys_call.c

- sys_call.c contains all functions pertaining to dispatching in the OS. These functions alter the currently running processes and queues.

sys_call()

- struct context* sys_call(context* ctx)
- Author: Chris Jones, Evan Humphrey
- This function takes the currently running process in the queue and performs the specified operation.
- Parameters:
 - context ctx* is the pointer to the context of the next process to be selected from the queue
- Return Values:
 - This function returns a pointer to the context of the next process in the queue.

find_first_ready()

- pcb* find_first_ready(void)
- Author: Chris Jones
- This function finds the first process in the ready queue and returns a pointer to it.
- Parameters:
 - There are no parameters for this function
- Return Values:
 - This function returns a pointer to the first PCB in the ready queue.

I0_Scheduler()

- `int I0_Scheduler(context* ctx, int op_code)`
- Author: Tanner Forbes
- This function processes input and output requests from the device given
- Parameters:
 - `context* ctx` is the context of the process receiving I/O requests
 - `int op_code` is the operation to perform on characters read or received from the device
- Return Values:
 - Returns an integer status code
 - 0 on success
 - Non-zero on failure

I0_Completion()

- `void I0_Completion(context* ctx, iocb* current_iocb)`
- Author: Tanner Forbes
- This function removes a process from the blocked state and returns it to the ready state and prepares it for the next I/O operation
- Parameters:
 - `context* ctx` is the context of the process to be changed
 - `iocb*` is the I/O operation to be removed and replaced
- Return Values:

- None

includes/mpx/sys_call.h

This file contains all functions pertaining to process dispatching.

Global Variables:

pcb* CURRENT_PCB

This pointer points to the currently running process in the system, and is updated during every context switch.

ctx* GLOBAL_CTX

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

pcb* nextPCB

This pointer is used to track the position of the next process in the queue.

struct context* sys_call(context* ctx)

- This function returns a pointer to the next process to execute in the system.

pcb* find_first_ready(void)

- This functions finds and returns the first process in the ready queue.

includes/context.h

This file contains all structures and functions relating to the context of a process.

Structs:

- **context**
 - Segment Registers:
 - int CS
 - int DS
 - int ES
 - int FS
 - int GS
 - int SS
 - Status Control Registers:
 - int EIP
 - int EFLAGS
 - General Purpose Registers:
 - int EAX
 - int EBX
 - int ECX
 - int EDX
 - int ESI
 - int EDI
 - int EBP
 - int ESP

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:

pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,

BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
 - char* name
 - int class
 - int priority
 - int state
 - unsigned char* stack
 - unsigned char* stack_location
 - struct pcb* next_node
 - struct pcb* prev_node

- **queue**
 - int queue_type
 - pcb* head
 - pcb* tail

Function Prototypes:

queue* return_queue(int class)

- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)

- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)

- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)

- This function frees up the memory allocated to a process control block

`pcb* pcb_find(const char * name)`

- This function finds a given process in the system in either queue

`void pcb_insert(pcb* pcbPtr)`

- This function inserts a new PCB into its corresponding queue in the system

`int pcb_remove(pcb* pcbPtr)`

- This function removes the specified PCB from the queue but does not delete it from the system

alarm.c

- alarm.c contains all functions pertaining to alarm dispatch and monitoring.

alarm_process()

- void alarm_process(void)
- Author: Chris Jones
- This function monitors the currently running alarms in the queue and makes sure they trigger when needed.
- Parameters:
 - None
- Return Values:
 - This function has no return values

create_alarm()

- int create_alarm(char* message, char* time)
- Author: Chris Jones
- This function takes parameters from the user and creates an alarm process that is placed into the process queue
- Parameters:
 - message is the message to be displayed to the user when the alarm is triggered
 - time is the system time specified by the user for when the alarm should trigger
- Return Values:
 - This function returns a status code based on the operation

- -1 for failure
- 0 for success

include/mpx/time.h

This file contains all functions relating to the real time clock (RTC)

Function Prototypes:

int get_time(void)

- This function prints out the current time in Eastern Standard Time using the real time clock

int set_time(char buf[])

- This function utilizes the real time clock registers to set the time in the system

int get_date(void)

- This function retrieves the date using the real time clock and prints it to the terminal

int set_date(char buf[])

- This function utilizes the real time clock registers to set the current date within the system

int get_hour(void)

- This function returns an integer representation of the current hour in the system.

int get_minute(void)

- This function returns an integer representation of the current minute in the system.

int get_second(void)

- This function returns an integer representation of the current second in the system.

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:

pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,
BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
 - char* name
 - int class
 - int priority
 - int state
 - unsigned char* stack
 - unsigned char* stack_location
 - struct pcb* next_node
 - struct pcb* prev_node
- **queue**
 - int queue_type
 - pcb* head
 - pcb* tail

Function Prototypes:

`queue* return_queue(int class)`

- This function returns a pointer to the desired queue head

`pcb* pcb_setup(char* name, int class, int priority)`

- This function allocates memory for a new PCB and sets all of the required fields within the structure

`pcb* pcb_allocate(void)`

- This function dynamically allocates memory for a new PCB struct

`int pcb_free(struct pcb* free_pcb)`

- This function frees up the memory allocated to a process control block

`pcb* pcb_find(const char * name)`

- This function finds a given process in the system in either queue

`void pcb_insert(pcb* pcbPtr)`

- This function inserts a new PCB into its corresponding queue in the system

`int pcb_remove(pcb* pcbPtr)`

- This function removes the specified PCB from the queue but does not delete it from the system

`include/sys_req.h`

This file contains all system request functions and constants

Enums:

`op_code`

`EXIT,`

IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- INVALID_OPERATION
- INVALID_BUFFER
- INVALID_COUNT

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:

`int sys_req(op_code op, ...)`

- This function requests a kernel operation within the system to perform an action based on the input parameters.

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

`char* strcat(char* des, const char* src)`

- This function concatenates two strings together into a destination string

`includes/mpx/sys_call.h`

This file contains all functions pertaining to process dispatching.

Global Variables:

`pcb* CURRENT_PCB`

This pointer points to the currently running process in the system, and is updated during every context switch.

`ctx* GLOBAL_CTX`

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

`pcb* nextPCB`

This pointer is used to track the position of the next process in the queue.

`struct context* sys_call(context* ctx)`

- This function returns a pointer to the next process to execute in the system.

`pcb* find_first_ready(void)`

- This functions finds and returns the first process in the ready queue.

mcb.c

- mcb.c is where the execution of all kernel level commands pertaining to Memory Control Blocks occur

Constant(s):

MCB_ARR_SIZE

- This integer value represents the size of the stack for a memory control block

Global Variables:

char mcb_arr

- This array is the stack containing all information pertaining to the MCB

int mcb_offset

- This value contains an offset that cushions the MCB between the stack and the struct

list MCB_LIST

- This list struct contains the head and tail nodes of the MCB list

char str

- This is the buffer used to print the results of itoa()

initialize_heap()

- void initialize_heap(size_t size)
- Authors: Evan Humphrey
- This function allocates the desired amount of heap space requested by the system into one contiguous block
- Parameters:
 - size_t size is the amount of bytes to allocate

- Return Values:
 - This function does not return any values

mcb_setup()

- mcb* mcb_setup(unsigned int start_addr, int size)
- Authors: Chris Jones
- This function takes the desired parameters for a new MCB and returns a pointer to the newly created struct
- Parameters:
 - unsigned int start_addr is the start address of the free or allocated block of memory
 - int size is the size of the free or allocatee block of memory
- Return Values:
 - This function returns a pointer to the newly created MCB

alloc_mcb()

- mcb* alloc_mcb(void)
- Author: Tanner Forbes
- This function prints a list of each available command
- Parameters:
 - None
- Return Values:
 - This function returns a pointer to the newly allocated memory for an MCB

return_list()

- `list* return_list(void)`
- Authors: Chris Jones
- This function fetches a pointer to the head of the MCB list
- Parameters:
 - None
- Return Values:
 - This function returns a pointer to the head of the MCB list

allocate_memory()

- `void * allocate_memory(size_t size)`
- Authors: Izaak Whetsell
- This function allocates and sets up a new block of memory with the passed in size and assigns an MCB to it
- Parameters:
 - `size_t size` is the size of memory that needs to be allocated
- Return Values:
 - This function returns a pointer to the newly allocated block of memory in the system

free_memory()

- `int free_memory(void *addr)`
- Authors: Tanner Forbes

- This function frees any allocated memory beginning at a specified address
- Parameters:
 - Void *addr is the pointer to the start of the allocated memory block
- Return Values:
 - This function returns an integer based on the status of the operation
 - 0 on success
 - 1 on failure

mcb_insert()

- Void mcb_insert(mcb* new_mcb)
- Authors: Chris Jones
- This function inserts the given MCB into the list
- Parameters:
 - mcb* new_mcb is the pointer to the MCB we would like to insert
- Return Values:
 - None

mcb_remove()

- void mcb_remove(mcb* current_mcb)
- Authors: Chris Jones
- This function removes the given MCB from the list
- Parameters:

- `mcb* current_mcb` is the pointer to the MCB we would like to remove
- Return Values:
 - None

`mcb_find()`

- `mcb* mcb_find(unsigned int start_addr)`
- Authors: Chris Jones
- This function finds an MCB from its given start address
- Parameters:
 - `unsigned int start_addr` is the start address of the memory that the MCB is associated with
- Return Values:
 - This function returns a pointer to the found MCB
 - NULL if not found

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

`int contains(const char *str1, const char *str2)`

- This function indicates whether `str1` contains `str2`

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

`char* strcat(char* des, const char* src)`

- This function concatenates two strings together into a destination string

include/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:

`void *sys_alloc_mem(size_t size)`

- This function dynamically allocates memory in the system

`int sys_free_mem(void *ptr)`

- This function frees dynamically allocated memory located within the system

`void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))`

This function installs user-supplied heap-management functions

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Global Variables:

`char color[10]`

This variable tracks the color set by the user for their CLI theme

Function Prototypes:

`int comexec(char buf[])`

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

`int shutdown(void)`

- This function will terminate the command handling sequence in the system and shut down any processes executing

`int help(void)`

- This function will print a list of each available command from the current point in the system

`int version(void)`

- This function prints the compilation date and version of the system

`int clear_screen(void)`

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

- This function returns the desired color requested by a special variable in the program

`include/conversions.h`

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

This function takes an integer value and returns a char literal representation of its binary equivalent

user_mcb.c

- user_mcb.c is where the execution of all user level commands pertaining to Memory Control Blocks occur

Constant(s):

MCB_ARR_SIZE

-This integer value represents the size of the stack for a memory control block

show_mcb()

- void show_mcb(mcb* mcb)
- Authors: Chris Jones
- This function locates a desired Memory Control Block within the heap
- Parameters:
 - mcb* mcb is the pointer to the Memory Control Block that we want to find
- Return Values:
 - This function does not return any values

show_alloc_mem()

- void show_alloc_mem(void)
- Authors: Chris Jones
- This function prints out the list of allocated memory blocks within the heap
- Parameters:
 - None

- Return Values:
 - This function does not return any values

`show_free_mem()`

- `void show_free_mem(void)`
- Author: Chris Jones
- This function prints out the list of free memory blocks within the heap
- Parameters:
 - None
- Return Values:
 - This function does not return any values

`include/string.h`

This file contains all functions pertaining to string manipulation and identification

`int contains(const char *str1, const char *str2)`

- This function indicates whether str1 contains str2

`char* substr(char* str1, int index)`

- This function will return the substring of a passed-in string beginning at a starting index

`int isNumeric(char* str)`

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

void clear(char* arr)

- This function clears a string by setting all members to zero

char* strcat(char* des, const char* src)

- This function concatenates two strings together into a destination string

include/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:

```
void *sys_alloc_mem(size_t size)
```

- This function dynamically allocates memory in the system

```
int sys_free_mem(void *ptr)
```

- This function frees dynamically allocated memory located within the system

```
void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))
```

- This function installs user-supplied heap-management functions

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Global Variables:

```
char color[10]
```

This variable tracks the color set by the user for their CLI theme

Function Prototypes:

`int comexec(char buf[])`

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

`int shutdown(void)`

- This function will terminate the command handling sequence in the system and shut down any processes executing

`int help(void)`

- This function will print a list of each available command from the current point in the system

`int version(void)`

- This function prints the compilation date and version of the system

`int clear_screen(void)`

- This function clears the screen that you are viewing in the terminal window and returns the cursor to the "home" position

`char* return_color(void)`

This function returns the desired color requested by a special variable in the program

`include/conversions.h`

This file contains a subset of C standard functions

Function Prototypes:

`char* custom_itoa(int num, char* str, int base)`

- This function converts a given integer value into an ASCII string

`char bcdToChar(unsigned char bcd)`

- This function converts a Binary Coded Decimal Value (BCD) into a character

`unsigned int intToBCD(unsigned int num)`

- This function converts a character into a Binary Coded Decimal Value (BCD)

`int Math_power(int num, int exponent)`

- This function calculates the value of an integer to the power of another integer

`unsigned int Hex2Dec(char *hex, int hex_size)`

- This function converts a hexadecimal value into an integer value

`char* Dec2Hex(unsigned int dec)`

This function takes an integer value and returns a char literal representation of its binary equivalent

`include/mcb.h`

This file contains all functions that execute kernel level commands relating to Memory Control Blocks

Structs:

- **mcb**
 - unsigned int start_addr
 - int size
 - struct mcb* next_node
 - struct mcb* prev_node
 - int status
 - char* name
- **list**
 - mcb* head
 - pcb* tail

Global Variables:

- list MCB_LIST contains all working MCBs in the system
- char str[100] is the string buffer for printing with itoa

Function Prototypes:

void initialize_heap(size_t size)

- This function is called to allocate the desired amount of bytes contiguously in the system for the heap

mcb* mcb_setup(unsigned int start_addr, int size)

- This function takes in multiple parameters for a desired memory control block, allocates memory for it, and returns a pointer to the newly initialized block with all desired values

void * allocate_memory(size_t size)

- This function will allocate a desired chunk of memory in the heap and assign an MCB to mark it. This function also combines any free blocks in the system to preserve more memory for incoming processes

`list* return_list(void)`

- This function returns a pointer to the head of the MCB list

`mcb* alloc_mcb(void)`

- This function allocates memory for a new MCB

`int free_memory(void *addr)`

- This function frees a block of allocated memory at the desired address

`void mcb_insert( mcb* new_mcb)`

- This function inserts a new MCB into the list

`void mcb_remove( mcb* current_mcb)`

- This function removes the desired MCB from the list

`mcb* mcb_find(unsigned int start_addr)`

- This function locates the desired MCB in the list using its start address

`void show_mcb(mcb* mcb)`

- This function prints out all details pertaining to the desired MCB you give a reference to

`void show_alloc_mem(void)`

- This function prints out all allocated memory blocks in the heap

`void show_free_mem(void)`

- This function prints out all free memory blocks in the heap

sys_call_isr.s

This file contains the code that performs the context switch of a process by using assembly to retrieve and store the values of currently running processes.

serial_isr.s

This file contains the code that performs the interrupts for I/O operations in the system.